

Computing a Matrix Minor in Boa: Select, Exclude, Multiply

```
import helper ;;

A =(9) [[2, 1, 4], [3, 5, 1], [7, 2, 6]] ;;

# Step 1: Remove a row and a column using exclusion indexing
# Negative subscripts exclude that element. To get the minor for (row, col) = (1, 2):
sub =(4) A[-1][-2] ;;
print "Submatrix after removing row 1, col 2:" ;;
print sub ;;

# Step 2: Determinant of the 2x2 sub (1-based, end-inclusive)
det =(8) sub[1][1] * sub[2][2] - sub[1][2] * sub[2][1] ;;
print "Minor (1,2) determinant:" ;;
print det ;;

# Step 3: Double-check your submatrix shape with matrix multiply
# Use identity matrix – the product should be the sub itself.
identity =(8) [[1, 0], [0, 1]] ;;
check =(8) sub @ identity ;;
print "Identity check (should match sub):" ;;
print check ;;

# Step 4: All nine minors in a loop – note the ;; after for headers
for i in [1, 2, 3]::;
  for j in [1, 2, 3]::;
    sub =(4) A[-i][-j] ;;
    det =(8) sub[1][1] * sub[2][2] - sub[1][2] * sub[2][1] ;;
    print "Minor(", i, ",", j, ") =", det ;;
  ;;
;;

# Step 5: Watch out for 1x1 matrices – exclusion gives an empty list
single =(4) [[5]] ;;
print "Excluding row 1 from 1x1:" ;;
print single[-1] ;; # prints []
# Then indexing would raise IndexError – guard with a dimension check.

# Allocation notes: every new list needs an =(N) before assignment,
# and @ returns a fresh object that also needs allocation.
# Use helper.memstats() to track if you're over-allocating.
```